SOLUTION ARCHITECTURE REPORT

Real-Time Fraud Detection Platform

A cloud-native, event-driven and observable design for screening 1,000 transactions per second

Prepared by	Saket Khopkar, AI Software Engineer
Audience	Bank architecture review board, engineering, risk and compliance
Version / date	1.0 | November 2026
Status	Proposed architecture for approval

Headline targets. Every transaction scored within 1 second (p99) of acceptance. Fraud alerts visible to analysts within 3 seconds (p99). Sustained 1,000 transactions per second, load-tested to 2,000. Zero loss of any acknowledged transaction, with every event captured immutably and replayable for compliance. Compute scales with demand instead of being paid for at peak.

Contents of this report: requirements and assumptions, patterns and rationale, solution architecture, microservice decomposition, event design, end-to-end flow, compliance replay, frontend, service mesh and security, technology stack, scalability, resilience, observability and SLOs, peak-load planning, deployment and migration, trade-offs, and a requirements traceability appendix.

Contents

1. Executive summary . . . 4

2. Problem analysis and requirements . . . 4

2.1 From business problems to technical objectives . . . 4

2.2 Assumptions and scope . . . 4

3. Architectural approach and patterns . . . 5

4. Solution architecture . . . 6

4.1 What each part of the architecture achieves . . . 7

5. Functional decomposition: microservices . . . 7

5.1 Why these boundaries . . . 8

5.2 Inter-service communication and coupling . . . 8

6. Event-driven design . . . 8

6.1 Event catalogue . . . 8

6.2 Event envelope and example schemas . . . 11

6.3 Topic and partitioning design . . . 11

6.4 Delivery semantics, ordering and failure handling . . . 11

6.5 Triggers for each microservice . . . 11

6.6 Deliberate deviations from the event-driven pattern . . . 11

7. End-to-end transaction flow . . . 12

7.1 Latency budget . . . 12

7.2 What happens when a stage is slow or failing . . . 12

8. Compliance: capture and replay . . . 14

8.1 Capture . . . 14

8.2 Replay . . . 14

8.3 Retention versus privacy . . . 15

8.4 Archive sizing (illustrative) . . . 15

9. Frontend architecture . . . 15

9.1 How data is fetched . . . 15

9.2 Why a Backend for Frontend . . . 16

9.3 Caching and performance . . . 16

9.4 Frontend SLO . . . 16

10. Service mesh and security . . . 16

10.1 Role of the service mesh . . . 16

10.2 Defence in depth . . . 17

11. Technology stack . . . 17

12. Scalability and elasticity . . . 18

12.1 Scaling individual microservices . . . 18

12.2 Scaling event messaging . . . 19

12.3 Scaling analytic workloads . . . 19

12.4 Capacity summary . . . 19

13. Resilience 20

13.1 Single points of failure and mitigations 20

13.2 Graceful degradation 20

13.3 Data loss and recovery 20

14. Observability 21

14.1 How logs, metrics and traces are collected 21

14.2 How logs, metrics and spans are correlated 22

14.3 Synthetic end-to-end probes 23

14.4 Key metrics and service level objectives 23

14.5 Service level objectives 23

14.6 Alerting strategy 23

15. Peak usage profiles and compliance events 24

16. Deployment in practice and migration 24

16.1 Reference deployment on AWS 24

16.2 Migration from the legacy batch system 25

17. Critical evaluation: trade-offs and risks 25

17.1 Principal risks and challenges 26

18. Conclusion and next steps 26

Appendix A: Requirements traceability 26

Appendix B: Glossary 27

1. Executive summary

The bank has migrated its services to the cloud, but fraud detection still runs as an overnight batch. New regulation requires real-time detection, and the business wants better detection of fraudulent transactions. This report proposes a replacement: an **event-driven streaming platform** in which every transaction flows through ingestion, feature enrichment, machine-learning scoring and a decision step, and fraud alerts are pushed to analysts within seconds.

Three ideas shape the design:

- **The event log is the backbone and the system of record.** Apache Kafka carries every event between services, and an archive service writes every event to immutable (WORM) storage. This single decision delivers real-time processing, decoupling between services, and the compliance requirement that every transaction is captured and replayable.
- **Stateless, partitioned services scale elastically.** Each microservice scales on the metric that matters to it (mostly Kafka consumer lag), so capacity follows demand and cost follows capacity. The messaging tier is the one component sized for peak.
- **Observability and resilience are designed in, not added.** OpenTelemetry traces, metrics and logs are correlated by shared identifiers, service level objectives drive alerting, and every single point of failure has a documented mitigation and recovery path.

The report covers ten deployable components (Section 5), ten event types (Section 6), the end-to-end flow with a latency budget (Section 7), compliance replay (Section 8), the analyst frontend (Section 9), the technology stack (Section 11), scaling (Section 12), resilience (Section 13) and observability with measurable SLOs (Section 14). Section 17 evaluates the trade-offs honestly, and Appendix A maps every requirement to the section that satisfies it.

2. Problem analysis and requirements

2.1 From business problems to technical objectives

The bank's legal requirements are business statements. To design against them, each is translated into a measurable technical objective.

Business requirement	Technical objectives	Addressed in
Real-time analysis of transactions	Streaming (not batch) processing. End-to-end scoring latency under 1 s at p99. Each pipeline stage has a latency budget. Work is partitioned for parallelism.	Sections 5, 7
Transactions captured and replayable for compliance	Durable write before acknowledgement. Immutable, ordered, time-indexed archive of all events. Deterministic replay in isolation from live traffic.	Section 8
Peak of 1,000 transactions per second	Horizontally scalable, partitioned consumers. Capacity target of 2x peak (2,000 TPS) proven by load test. Back-pressure instead of collapse.	Sections 12, 15
Elastic scaling without paying peak cost all the time	Stateless services that autoscale on consumer lag. Node autoscaling. Scheduled pre-scaling for known events. Spot capacity for batch work.	Section 12
Real-time alerting for analysts via a frontend	Server push over WebSocket through a Backend for Frontend. Alert visible within 3 s at p99. Safe reconnect and catch-up.	Section 9
Observability and alerting to operationalise the system	Correlated traces, metrics and logs. Specific SLOs with burn-rate alerting. Model-health monitoring. Synthetic end-to-end probes.	Section 14
(Implicit) Financial-services standards	Zero-trust security, tokenised PII, audit trail, model governance, tested disaster recovery, explicit failure handling.	Sections 10, 13, 17

2.2 Assumptions and scope

- **Detection is near-real-time and advisory, not an inline gate.** The bank's payment flow does not wait for the fraud verdict; analysts and downstream controls act on it within seconds. This keeps fraud-system outages from stopping payments. If inline blocking is later required, a synchronous decision API can be added in front of the same scoring components (discussed in Section 17).
- **Volume:** average of about 300 transactions per second, peak of 1,000, designed and tested to 2,000. These are planning assumptions to be confirmed with the bank.
- **Data:** payloads carry tokenised identifiers, never raw card numbers. Roughly 8 KB of events are produced per transaction across all stages.
- **Retention:** 7 years for the immutable archive (a regulatory parameter for compliance to confirm).
- **Cloud:** AWS is used as the reference platform. The patterns are portable to other providers (Section 16).

- **Model:** the ML model is supplied by data science. This report prescribes how the model is served, governed and monitored, not its algorithm. A gradient-boosted tree model is assumed as the baseline because it scores quickly on CPU and is explainable.
- **Alert volume:** a small fraction (about 0.1 to 1 percent) of transactions raise high-confidence alerts, so analysts see single-digit alerts per second at peak.

Out of scope: model algorithm design, visual design of the UI, and selection of a case-management product.

3. Architectural approach and patterns

The table lists each pattern used, where it appears, and its cost-benefit. Cost matters as much as benefit in a regulated environment, because every pattern adds something that must be operated, secured and audited.

Pattern	Where used	Benefit	Cost / risk
Event-driven architecture with a durable log	Kafka between all services	Decouples services in time. Absorbs bursts. Allows replay. Supports many independent consumers.	Eventual consistency. Harder to trace. Kafka must be operated well.
Microservices by business capability	Services 1 to 10 (Section 5)	Independent scaling and release. Fault isolation. Clear ownership.	Network calls, contract management, more to deploy and observe.
Event log as system of record (event sourcing, applied selectively)	Archive service and replay	Perfect audit trail. Any read model can be rebuilt. Replay for investigations.	Storage cost. Schema evolution must be managed for years.
CQRS (separate read model)	Alert Query & Case service	Fast, flexible analyst queries without loading the detection path.	Duplicated data. Read model is slightly behind the log.
Backend for Frontend (BFF)	Analyst BFF	UI-shaped APIs, fewer round trips, one place for caching and authorisation.	One more service to run. Must not accumulate business logic.
Service mesh	All service-to-service HTTP/gRPC	Mutual TLS, authorisation policy, retries, canary routing and uniform telemetry without code changes.	Latency and resource overhead. Operational complexity. Does not cover Kafka traffic.
Competing consumers and partitioned parallelism	All Kafka consumers	Linear horizontal scaling. Ordering preserved per account.	Parallelism capped by partition count. Hot keys possible.
Idempotent consumer, retry topics and dead-letter queue	Every consumer	Safe under at-least-once delivery. Poison messages never block the stream.	Needs deduplication logic and DLQ operations.
Caching (feature store, BFF cache, CDN)	Redis, CDN	Low latency and protection of backends.	Staleness and invalidation. Cache is never the source of truth.
Circuit breaker, timeout and bulkhead	Synchronous calls only	Stops cascading failure from a slow dependency.	Needs tuning and testing.
Strangler fig with shadow mode	Migration from the legacy batch system	Proves parity before cutover. Reversible.	Temporary double running cost.

4. Solution architecture

Figure 1 shows the complete solution: transactions enter through a hardened edge, flow through Kafka-connected services, and are captured and observed throughout.

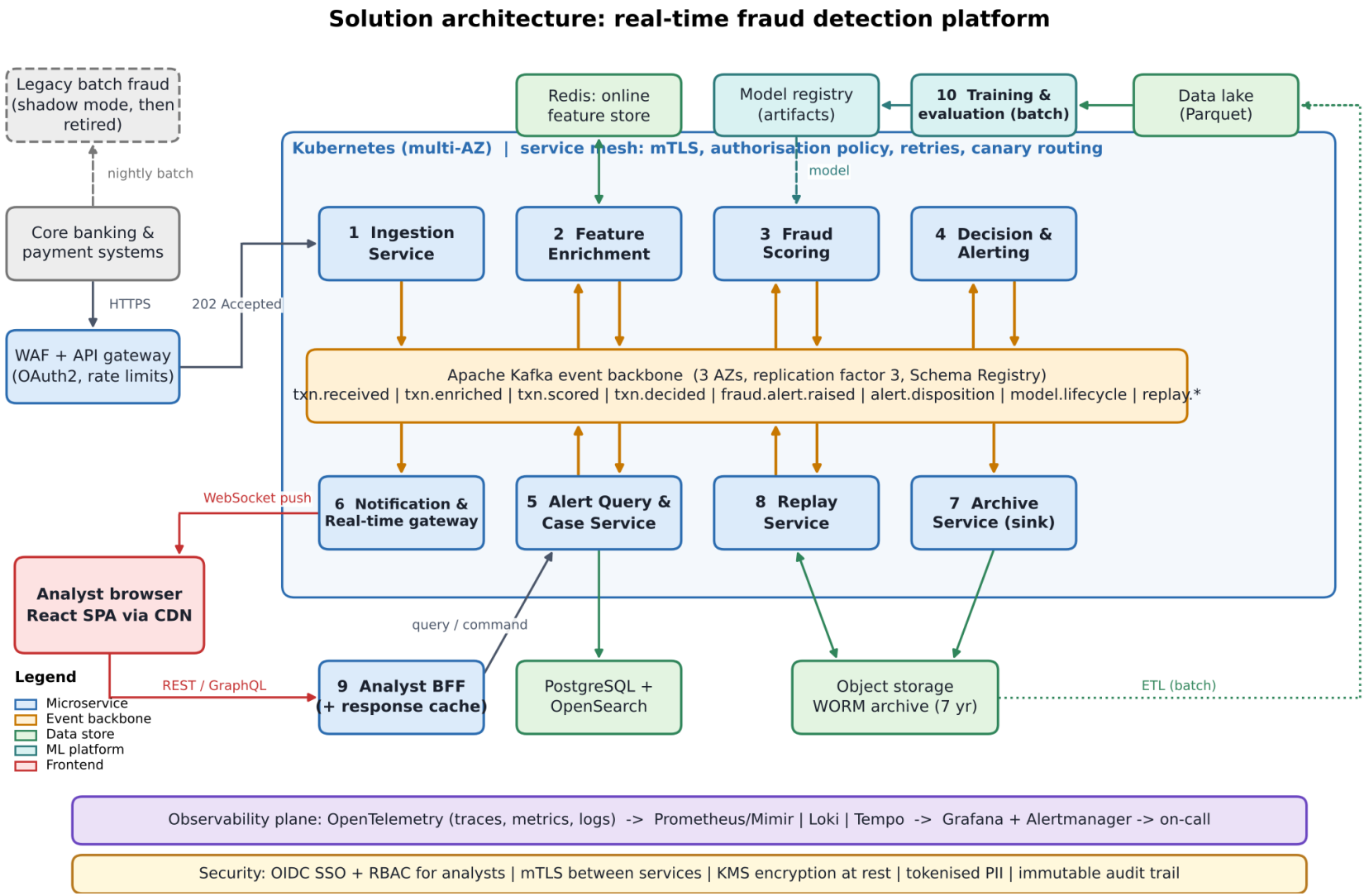


Figure 1: Solution architecture showing components, interactions and the event backbone.

4.1 What each part of the architecture achieves

Transactions enter at the left of Figure 1 through a hardened edge. The ingestion service publishes each one to Kafka, and a chain of services enriches, scores and decides, each reading from and writing to Kafka. Alerts are pushed to analysts. A parallel archive path captures everything for compliance. Observability and security span the whole platform.

Component	What it achieves
WAF and API gateway	A single authenticated, rate-limited entry point. Protects the platform from malformed or abusive traffic.
Kafka event backbone and Schema Registry	Durable, ordered, replayable transport. Contracts for every event type. Lets stages run at different speeds.
Services 1 to 4 (the detection path)	Turn a raw transaction into a decision in under a second.
Services 5, 6 and 9 (the analyst path)	Make results queryable and alerts visible in real time with a responsive UI.
Services 7 and 8 (the compliance path)	Capture every event immutably and replay any period for auditors, away from live traffic.
ML platform (service 10)	Trains, evaluates, registers and promotes models offline, learning from analyst feedback.
Observability plane and security layer	Operate, measure and protect the platform end to end.

5. Functional decomposition: microservices

The requirements are decomposed into ten components, each with one primary responsibility, a defined trigger, and its own data. Services are split along the boundaries where scaling behaviour, rate of change or ownership differ.

Service	Responsibility	Trigger	Inputs and outputs	Owns
1 Ingestion	Single front door for bank payment systems. Authenticates, validates the schema, de-duplicates on the idempotency key, stamps correlation id and trace context, durably publishes, then acknowledges.	HTTPS/gRPC request from a bank payment system	In: transaction JSON. Out: HTTP 202, 4xx, 429 or 503; event txn.received	Short-lived de-duplication cache
2 Feature Enrichment	Computes behavioural and contextual features: velocity, amount versus history, location and device change, merchant risk, account risk flags.	Event txn.received; event alert.disposition (updates account risk)	In: txn.received. Out: txn.enriched (transaction plus feature vector and feature versions)	Online feature state (Redis), rebuildable from Kafka
3 Fraud Scoring	Runs the promoted ML model. Returns a score, calibrated confidence and top contributing factors, and records the model version.	Event txn.enriched; event model.lifecycle (load new model)	In: txn.enriched, model artifact. Out: txn.scored	Nothing durable; stateless
4 Decision & Alerting	Applies versioned business policy (thresholds, rules, allow-lists) to scores. Classifies FRAUD, REVIEW or CLEAR and raises alerts for high confidence.	Event txn.scored	In: txn.scored, policy config. Out: txn.decided, fraud.alert.raised	Versioned policy configuration
5 Alert Query & Case	Read model and analyst workflow: search, assignment, disposition (true or false positive), reporting.	Events txn.decided and fraud.alert.raised; HTTP queries and commands from the BFF	In: events, commands. Out: query results; event alert.disposition	PostgreSQL (alerts, cases), OpenSearch (search model)
6 Notification & Real-time Gateway	Pushes alerts to connected analyst browsers and external channels (pager, email, case tools). Manages connections, sequence numbers and resume.	Event fraud.alert.raised; WebSocket connect	In: fraud.alert.raised. Out: pushed messages, external notifications	Connection registry, short resume buffer
7 Archive	Writes every event immutably, in order, partitioned by event time. Produces hash manifests and reconciliation counts.	Continuous subscription to all topics	In: all events. Out: WORM objects, reconciliation metrics	WORM object storage
8 Replay	Executes approved replay requests: reads an archive range, re-publishes it to isolated topics, collects results and produces an audit report.	Event replay.command (from Compliance API); event replay.results	In: command, archive. Out: txn.replay.received, audit report	Replay job state, report store
9 Analyst BFF	Tailors APIs to the analyst UI: aggregates calls, shapes payloads, caches, enforces authorisation, exposes compliance endpoints.	HTTPS request from the SPA	In: REST/GraphQL calls. Out: JSON; commands to services 5 and 8	Response cache

Service	Responsibility	Trigger	Inputs and outputs	Owns
10 ML Platform	Offline: builds training sets from the lake and analyst labels, trains, evaluates, registers and promotes models, and monitors drift.	Schedule, drift alert, or enough new labels	In: lake data, dispositions. Out: model artifacts, event model.lifecycle	Data lake, model registry

5.1 Why these boundaries

- **Enrichment is separate from scoring.** Enrichment is I/O and state heavy; scoring is CPU heavy. They scale on different signals and change at different rates (feature engineering versus model releases). Features can also feed several models.
- **Decision is separate from scoring.** Thresholds and rules are risk-policy decisions owned by the business. They can change without redeploying a model, and a replay can apply a different policy to the same scores.
- **Archive is separate and never on the critical path.** Compliance capture must not slow detection, and a detection outage must not stop capture.
- **Notification is separate from Alert Query.** Push fan-out scales with the number of connected analysts, not with the event rate.
- **Not decomposed further.** Splitting by individual fraud rule or per feature would add network hops and operational burden with little benefit at this scale.

5.2 Inter-service communication and coupling

Services communicate asynchronously through Kafka by default. Synchronous calls exist only where a caller needs an immediate answer (listed in Section 6.6). The coupling that remains is deliberate and managed:

Type of coupling	Assessment	How it is managed
Temporal (must both be up at once)	Low. A slow or restarting consumer only builds lag in Kafka.	Durable log, lag-based autoscaling, lag alerts.
Contract (shared event schemas)	Medium. This is the main coupling and is intentional.	Schema Registry with backward-compatible evolution enforced in CI, consumer-driven contract tests, versioned topics (v1, v2) for breaking changes.
Data (shared databases)	None. Each service owns its store. Only Enrichment touches the feature store.	Services receive data in events (event-carried state), never by reading another service's database.
Platform (shared Kafka cluster)	High. Kafka is a critical shared dependency.	Multi-AZ replication, quotas per client, DR mirror, strong operational ownership (Section 13).
Deployment	Low. Services release independently.	Backward-compatible schemas, canary rollouts via the mesh.

6. Event-driven design

6.1 Event catalogue

Each event states what happened, when it is published and why it exists. Figure 2 shows publishers and consumers for every event type.

Event (topic)	Publisher	When it is published	Why it exists	Consumers
TransactionReceived txn.received.v1	Ingestion	After validation and durable acceptance	Starts the pipeline and is the system-of-record entry for the transaction	Enrichment, Archive
TransactionEnriched txn.enriched.v1	Enrichment	After features are computed	Carries the exact feature vector used, which supports explainability and replay	Scoring, Archive
TransactionScored txn.scored.v1	Scoring	After model inference	Records raw model output separately from business policy	Decision, Alert Query, Archive
TransactionDecided txn.decided.v1	Decision	After thresholds are applied (every transaction)	Gives reporting a complete decision history and decouples policy from the model	Alert Query, Archive
FraudAlertRaised fraud.alert.raised.v1	Decision	Only when the decision is FRAUD with high confidence	Low-volume, high-priority stream so the push gateway never filters the full firehose	Notification, Alert Query, Archive
AlertDispositionRecorded alert.disposition.v1	Alert Query & Case	When an analyst confirms or rejects an alert	Closes the loop: updates account risk and becomes training labels	Enrichment, ML platform, Archive

Event (topic)	Publisher	When it is published	Why it exists	Consumers
ModelLifecycleChanged model.lifecycle.v1	ML platform	When a model is promoted or rolled back	Tells scoring which model to load (event carries a pointer, not the artifact)	Scoring, Archive
ReplayRequested replay.command.v1	Compliance API	When an auditor submits an approved replay	Starts a replay job asynchronously	Replay
TransactionReplayed txn.replay.received.v1	Replay	As archived events are re-published	Feeds the isolated replay-mode pipeline	Replay-mode pipeline
ReplayResultProduced replay.results.v1	Replay-mode pipeline	As replayed decisions are produced	Collects outcomes for the audit report	Replay

Every consumer also has retry and dead-letter topics (topic.retry and topic.dlq) described in Section 6.4.

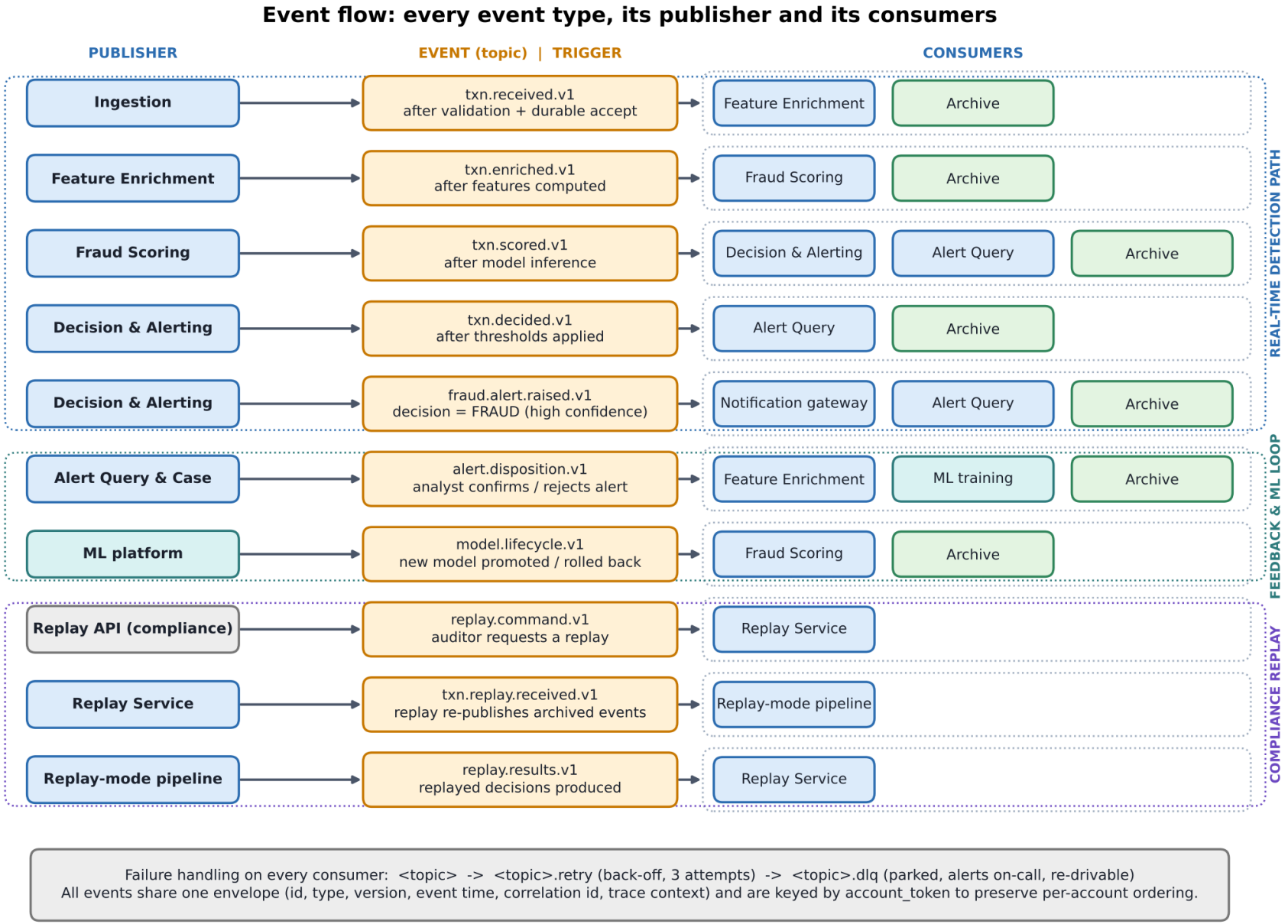


Figure 2: Event flow diagram: each event type with its publisher, trigger and consumers.

6.2 Event envelope and example schemas

All events share one envelope so that tracing, auditing and replay work identically for every type. Schemas are defined in Avro or Protobuf and registered in the Schema Registry; JSON is shown here for readability. Payloads carry tokens, never raw card numbers or names.

```
{ "event_id": "0b6f...e1",            "event_type": "txn.scored",       "schema_version": "1.2",
  "occurred_at": "2026-11-06T09:14:22.418Z",   // event time, set at the source
  "published_at": "2026-11-06T09:14:22.601Z",
  "correlation_id": "txn-8c41...",     "causation_id": "event_id of the triggering event",
  "traceparent": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01",
  "producer": "fraud-scoring@1.14.2",  "replay_id": null,
  "payload": { ... }                   // type-specific, see below
}
```

```
txn.received   payload: { transaction_id, idempotency_key, source_system, account_token, instrument_token,
                          merchant_id, merchant_category, amount: {value, currency}, channel,
                          location: {country, geohash}, device_token, transaction_time }
txn.scored     payload: { transaction_id, model_id, model_version, score (0..1), confidence,
                          top_factors: [{feature, contribution}], feature_vector_hash, degraded: false }
fraud.alert    payload: { alert_id, transaction_id, account_token, score, confidence, severity,
                          reasons[], recommended_action, policy_version, model_version, decided_at }
```

6.3 Topic and partitioning design

- **Partition key is the account token.** All events for an account land on one partition, so per-account ordering holds. Velocity features depend on this.
- **48 partitions** on the main topics. That is more than twice the maximum expected consumer count (20), leaving room to scale without repartitioning, which is disruptive.
- **Hot keys** (very large corporate accounts) are watched through partition skew metrics and mitigated by adding the instrument token to the key where ordering across instruments is not required.
- **Retention:** 7 days hot (with tiered storage to extend cheaply) so consumers can recover from outages and replays of recent data need no archive access. The archive holds the long-term record.
- **Naming:** domain.entity.version, for example txn.scored.v1. Breaking schema changes create a new version and both run during migration.

6.4 Delivery semantics, ordering and failure handling

- **At-least-once delivery with idempotent consumers.** Producers are idempotent. Consumers commit offsets only after processing and de-duplicate on event_id or transaction_id, so redelivery is harmless. Kafka transactions are used where a stream processor must read, write and commit atomically.
- **Event time, not processing time.** Features and windows use occurred_at, which makes results reproducible when events are replayed later.
- **Retry, then park.** A failed message goes to topic.retry with back-off (three attempts), then to topic.dlq. A message in a DLQ raises an alert, never blocks the partition, and can be re-driven after a fix.
- **Schema safety.** Producers cannot publish an incompatible schema, so a bad deployment cannot silently poison downstream consumers.

6.5 Triggers for each microservice

Every service has an explicit trigger (see the Trigger column in Section 5): Ingestion by an inbound request; Enrichment by txn.received; Scoring by txn.enriched; Decision by txn.scored; Alert Query by txn.decided and fraud.alert.raised; Notification by fraud.alert.raised; Archive by every topic; Replay by replay.command; the BFF by an inbound request; the ML platform by a schedule, a drift alert or accumulated labels.

6.6 Deliberate deviations from the event-driven pattern

Pure event-driven design is not always right. These interactions are intentionally different, and why:

Interaction	Style used	Why it deviates
Bank payment system to Ingestion	Synchronous request/response	The source needs an acknowledgement, idempotency and a clear back-pressure signal (202, 429, 503) so it can buffer and retry. Ack is sent only after the Kafka write is durable.
BFF to Alert Query for UI queries	Synchronous REST/GraphQL	A user waiting on a screen needs an immediate answer from a read model.

Interaction	Style used	Why it deviates
Analyst disposition	Synchronous command, then event	The analyst needs confirmation and strong consistency from PostgreSQL. The event is published after the write.
Alerts to the browser	WebSocket push	Kafka is not exposed to browsers. The gateway bridges events to push connections.
Feature lookup in Enrichment	Synchronous Redis access	A sub-millisecond state lookup inside one service's own boundary. Not a cross-service call.
Model distribution	Event announces, artifact fetched from object storage	Model files are too large for events. The event carries a pointer and checksum.
Telemetry (traces, metrics, logs)	Push/pull outside the event backbone	Telemetry overload or failure must never affect transaction processing.
Reconciliation against the source ledger	Scheduled batch job	A periodic completeness check is naturally batch. It catches loss that events cannot detect.

7. End-to-end transaction flow

Figure 3 (on the page after this section's text) follows one transaction from the bank's payment system to the analyst's screen. Only the first and last hops are synchronous. Everything in between is a chain of events on Kafka, and each stage has a latency budget so that a slow stage is visible and attributable.

7.1 Latency budget

Stage	Typical (ms, cumulative)	p99 budget (ms)	Notes
1 Ingestion validates, writes durably, acknowledges	50	100	Measured by the ingestion SLO. Includes the Kafka write with acks=all.
2 Kafka hop and Enrichment (feature lookup)	110	200	Redis lookups complete in about 1 ms. Most time is the hop and computation.
3 Kafka hop and Scoring (model inference)	190	250	Embedded model, about 10 ms of CPU, plus explanation of top factors.
4 Kafka hop and Decision (policy)	225	100	Pure in-memory policy evaluation.
5 Kafka hop and push to the browser	275	200	Gateway holds the WebSocket and writes immediately.
6 Browser renders the alert	400	250	Renders are batched every 250 ms to keep the UI smooth under bursts.

The p99 budgets for stages 2 and 3 total 450 ms from acceptance to a scored event, against a 1 s SLO. Stages 2 to 6 total 1,000 ms from acceptance to an alert on screen, against a 3 s SLO. The margin of more than two times is deliberate: it absorbs queueing, garbage-collection pauses and short traffic bursts without breaching the objective.

7.2 What happens when a stage is slow or failing

- **Ingestion cannot reach Kafka:** it returns 503 and the source retries with the same idempotency key. No transaction is acknowledged without being durable.
- **A consumer falls behind:** events wait safely in Kafka. Lag rises, autoscaling reacts, and the lag alert fires if the backlog persists.
- **Redis is unavailable:** Enrichment continues with default features and marks events degraded, so scoring still happens and the degraded ratio is monitored (Section 13).
- **A message cannot be processed:** it is retried, then parked in the DLQ with an alert. The rest of the stream continues.

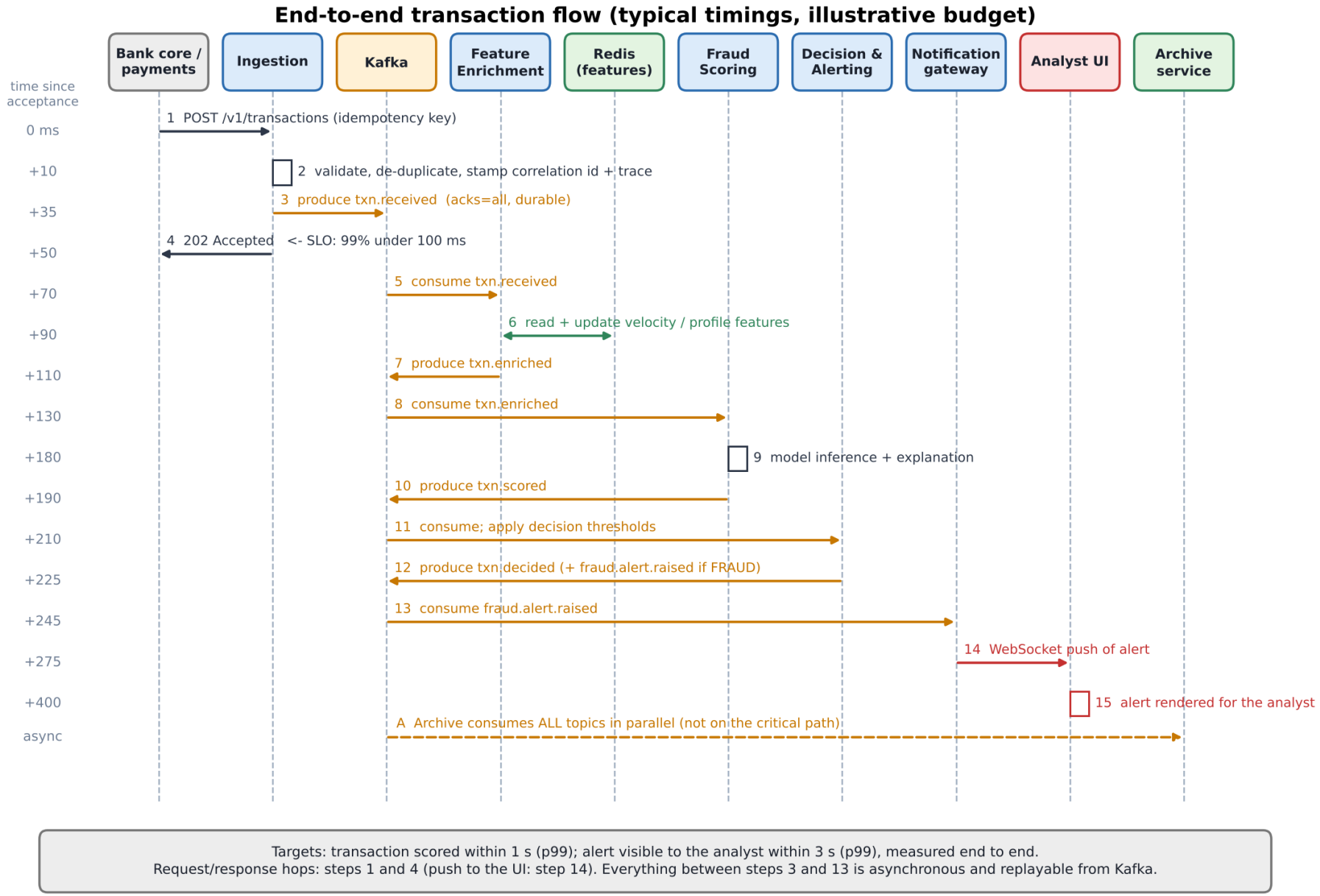


Figure 3: End-to-end transaction flow with typical timings (illustrative budget).

8. Compliance: capture and replay

Regulation requires that every transaction passing through the system is captured and can be replayed. The design meets this with a continuous, independent capture path and a controlled, isolated replay path (Figure 4).

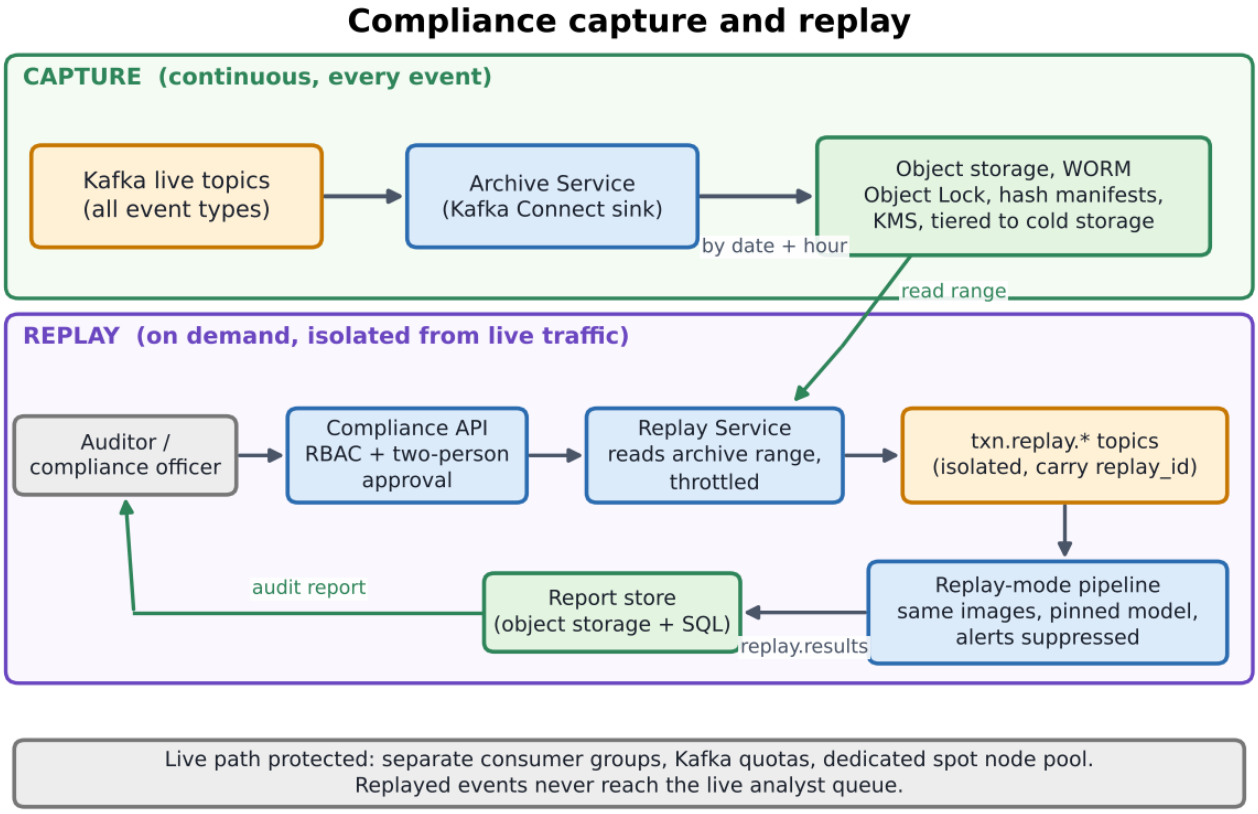


Figure 4: Compliance capture path and isolated replay path.

8.1 Capture

- **Complete.** The Archive service subscribes to every topic, so the archive holds not only transactions but the features, scores, decisions, alerts, analyst actions and model versions that produced each outcome.
- **Immutable.** Objects are written to WORM storage (object lock in compliance mode) with a retention period, versioning, KMS encryption and cross-region replication. Nobody, including administrators, can alter or delete them within retention.
- **Tamper-evident.** Each batch of objects has a hash manifest. A scheduled job verifies hashes and compares hourly event counts against the ingestion log and the bank's own ledger (reconciliation).
- **Queryable.** Data is stored in a columnar format partitioned by event date and hour, so an auditor's time-range request reads only the relevant objects.
- **Off the critical path.** A slow or failed archive builds lag in Kafka (7 days of buffer) and raises an alert. It never slows detection.

8.2 Replay

An auditor submits a replay request (time range, filters, and options) through the Compliance API. Replay requires role-based access and a second approver. The Replay service then re-publishes the archived events, in original order, to isolated replay topics (txn.replay.*). A replay-mode deployment of the same enrichment, scoring and decision images processes them and publishes results, which Replay assembles into an audit report.

- **Isolation protects live traffic.** Replay uses separate topics, consumer groups, Kafka quotas and a dedicated spot node pool. Replayed events carry a replay_id and can never reach the live analyst queue or trigger external notifications.
- **Determinism.** Results are reproducible because processing uses event time, the model version is pinned to the one recorded (or an explicitly chosen challenger), and the feature state is rebuilt from the ordered replayed events.

- **Two replay modes.** *Audit replay* reproduces what the system decided at the time. *Reprocess* applies a different model or policy to historical data to evaluate a change. Both are labelled in the report.
- **Throttled and scheduled.** Replay speed is capped so it cannot starve live processing, and large replays are scheduled outside peak hours.

8.3 Retention versus privacy

An immutable archive conflicts with the right to erasure. The design resolves this by never storing direct personal data in events. Events hold opaque tokens, and the mapping from token to person lives in a separate tokenisation vault with its own controls. If erasure is required, the vault entry is destroyed (crypto-shredding), which leaves the archive intact but no longer attributable. Legal and compliance teams must confirm this approach against each applicable regulation.

8.4 Archive sizing (illustrative)

Item	Assumption or result
Average volume	300 TPS, so about 26 million transactions per day
Event data per transaction (all stages)	About 8 KB
Raw volume	About 207 GB per day
Compressed columnar volume (assume 4:1)	About 52 GB per day, about 19 TB per year
7-year retention	About 130 TB, held mostly in infrequent-access and archive storage tiers

9. Frontend architecture

Analysts need a UI that shows new alerts almost instantly, stays responsive under bursts, and survives network interruptions. The frontend is a single-page React application served from a CDN. It talks to the platform only through the Analyst BFF and the Real-time gateway (Figure 5).

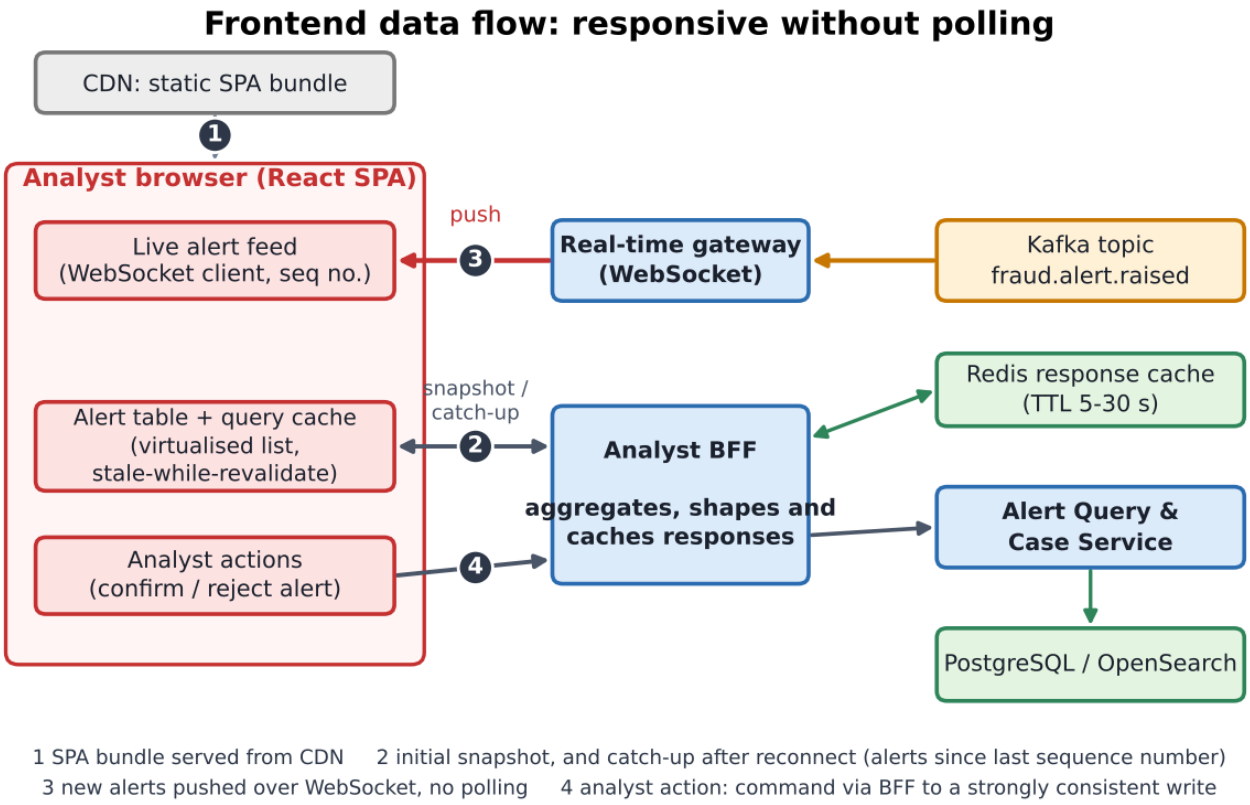


Figure 5: Frontend data flow: snapshot through the BFF, live updates by WebSocket.

9.1 How data is fetched

- **Snapshot, then stream.** On load the SPA fetches the current queue through the BFF with cursor-based pagination. It then receives new alerts over WebSocket. No polling.

- **Sequence numbers for safety.** Every pushed alert carries a per-stream sequence number. If the client sees a gap, or reconnects, it asks the BFF for alerts since its last sequence number and merges them. The UI is never silently stale.
- **Summary first, detail on demand.** The list shows compact alert summaries. Full transaction detail, feature contributions and history load when an analyst opens an alert.
- **Commands are synchronous and consistent.** Confirm, reject and assign go through the BFF to PostgreSQL, so analysts get immediate, correct confirmation. The resulting event updates everyone else's view.

9.2 Why a Backend for Frontend

The UI needs data from several services in one shape. A BFF aggregates them into a few UI-shaped calls, removes round trips, hides internal topology, and is a single place for caching and authorisation. It contains no fraud logic. The trade-off is one more service to operate, accepted because it keeps the core services free of UI concerns.

9.3 Caching and performance

Layer	What is cached	Lifetime and invalidation
CDN	Static SPA bundle and assets	Content-hashed file names, long expiry, invalidated by new deployments
Browser query cache	Recent query results	Stale-while-revalidate: show instantly, refresh in the background
BFF response cache (Redis)	Hot read models: queue counts, top open alerts, KPI tiles	5 to 30 seconds. Invalidated by incoming events for the affected queue
Pre-aggregation in Alert Query	Dashboard counters and trends	Updated incrementally from events, never computed on request

- **Smooth rendering under bursts:** virtualised lists, and UI updates batched about four times per second instead of per message.
- **Lean payloads:** compressed responses, only the fields the screen shows, HTTP/2 multiplexing.
- **Degraded modes:** automatic reconnect with exponential back-off and jitter. If WebSocket is blocked, the client falls back to short polling through the BFF and shows a status banner.
- **Security:** SSO via the bank's identity provider with MFA, short-lived tokens, role-based access to queues and fields, masking of sensitive data by default, and an audit log of what analysts view.

9.4 Frontend SLO

99 percent of GET requests from the analyst UI to the BFF complete in under 300 ms (Section 14.5). Browser-side timings (time to first alert, push-to-render) are reported by a lightweight real-user-monitoring beacon and correlated with backend traces through the same trace identifier.

10. Service mesh and security

10.1 Role of the service mesh

A service mesh (Istio, or a lighter alternative such as Linkerd) provides platform capabilities uniformly, so that individual teams do not each build them:

Capability	How it contributes
Mutual TLS and workload identity	**Security.** Every service-to-service call is encrypted and authenticated. Authorisation policy states which service may call which.
Retries, timeouts, circuit breaking	**Resilience.** Applied consistently to the synchronous calls (BFF to services, replay API). Prevents one slow dependency from cascading.
Traffic shifting and canaries	**Scalability and safety.** New service or model-serving versions receive a small share of traffic first and roll back automatically on SLO regression.
Uniform request metrics and trace headers	**Observability.** Golden-signal metrics (rate, errors, duration) for every service without code changes.

Limits and costs. The mesh adds a small latency and resource overhead per hop and a control plane to operate. It does not manage Kafka traffic, which is an opaque protocol; Kafka is protected by its own TLS, authentication and ACLs. The mesh data plane keeps working with its last configuration if the control plane is briefly unavailable, and the control plane runs with several replicas. Ambient or sidecar-less modes can reduce overhead if measurements show it matters.

10.2 Defence in depth

Layer	Controls
Identity and access	SSO with MFA for people. Workload identities for services. Least-privilege roles. Two-person approval for replay. Break-glass accounts that are audited.
Network	WAF and rate limiting at the edge. Private networking for data stores. Default-deny network policies between namespaces.
Data in transit and at rest	TLS everywhere (mTLS in the mesh, TLS and ACLs on Kafka). KMS-managed keys for every data store. Separate keys for the archive.
Data minimisation	Tokenised account, instrument and device identifiers. No raw PAN or names in events or logs. PII scrubbing in the telemetry pipeline.
Secrets and supply chain	Central secrets manager with rotation. Signed container images, vulnerability scanning, and pinned base images.
Auditability	Immutable archive for business events. Separate audit log for administrative and analyst actions.
Model security and governance	Signed model artifacts. Model registry with approval workflow. Recorded model version and explanation for every decision. Shadow and canary promotion before full rollout.

11. Technology stack

Choices are justified against the architectural requirements. Alternatives are noted because a different constraint, such as an existing enterprise standard, could reasonably change a choice.

Layer	Choice	Justification	Alternatives considered
Compute platform	Kubernetes (managed, multi-AZ)	Portable. Mature autoscaling (HPA, KEDA, node autoscaler). Fits the bank's cloud-native estate.	Serverless functions (cold starts and limited stream control), VMs (slower scaling)
Event backbone	Apache Kafka (managed service) with Schema Registry	Durable replayable log, ordering per key, consumer groups, very high throughput, rich connector ecosystem.	Cloud queues (no replay or ordering at scale), RabbitMQ (not a log), Kinesis or Pulsar (viable, narrower ecosystem or skills)
Stream processing	Kafka Streams (JVM services)	Stateful processing embedded in normal services. Scales with partitions. Exactly-once between topics.	Apache Flink (stronger for very complex windows, adds a cluster to run)
Service languages	Team choice per service (for example Java or Kotlin for stream services, Go for edge services)	The report prescribes patterns, not code. Stream services benefit from the JVM Kafka client; the edge benefits from low overhead.	Single language for all (simpler, less fit-for-purpose)
Model serving	Model embedded in the scoring service (ONNX or native runtime)	No extra network hop, simple scaling, deterministic latency. Gradient-boosted trees are fast on CPU and explainable.	Dedicated model server (use if GPU models or many models are introduced)
Online feature store	Redis (clustered, multi-AZ)	Sub-millisecond access. Derived data, so rebuildable from Kafka.	DynamoDB or Cassandra (durable but slower, unnecessary here)
Alert and case database	PostgreSQL (managed, multi-AZ)	Transactional integrity for analyst actions, relational audit history, point-in-time recovery.	NoSQL stores (weaker transactional guarantees)
Search read model	OpenSearch	Fast filtering and full-text search over alerts. Rebuildable from the log.	PostgreSQL full-text only (limited at scale)
Archive	Object storage with Object Lock (WORM), columnar files	Cheap, durable, immutable, queryable by serverless SQL engines.	Database archive (costly), tape (slow)
Data lake and analytics	Parquet or Iceberg tables, serverless SQL, ephemeral Spark	Separates analytic load from real-time stores. Pay per use.	Always-on warehouse (idle cost)
ML platform	Managed training and registry (for example SageMaker or MLflow)	Reproducible training, versioned artifacts, approval workflow.	Self-built pipeline (more work, more risk)
Frontend	React with TypeScript, TanStack Query, WebSocket	Mature ecosystem, strong typing, built-in stale-while-revalidate caching.	Server-rendered pages (less suited to live data)
API styles	REST/GraphQL for UI, REST/gRPC for ingestion	Well understood. GraphQL fits UI aggregation in the BFF.	gRPC-Web (extra tooling for browsers)

Layer	Choice	Justification	Alternatives considered
Service mesh	Istio (or Linkerd)	mTLS, policy, canaries and telemetry as a platform feature.	Library-based approach (inconsistent across languages)
Observability	OpenTelemetry, Prometheus or Mimir, Loki, Tempo, Grafana, Alertmanager	Open standards avoid lock-in. One collector pipeline for all signals. Strong correlation in Grafana.	Commercial APM (faster start, higher cost and lock-in)
Delivery	Terraform, GitOps (Argo CD), CI with contract tests	Reproducible environments and auditable change history.	Manual or script-based deployment

12. Scalability and elasticity

The target is 1,000 transactions per second at peak, proven to 2,000, with cost that tracks demand. The approach differs by workload type.

12.1 Scaling individual microservices

Services 1 to 4, 6, 7 and 9 are stateless or hold only rebuildable state, so they scale horizontally by adding replicas. Autoscaling uses the signal closest to the work (Figure 6). Consumers scale on **Kafka consumer lag** (using KEDA), which reacts to real backlog earlier than CPU does. Ingestion and the BFF scale on request rate and CPU.

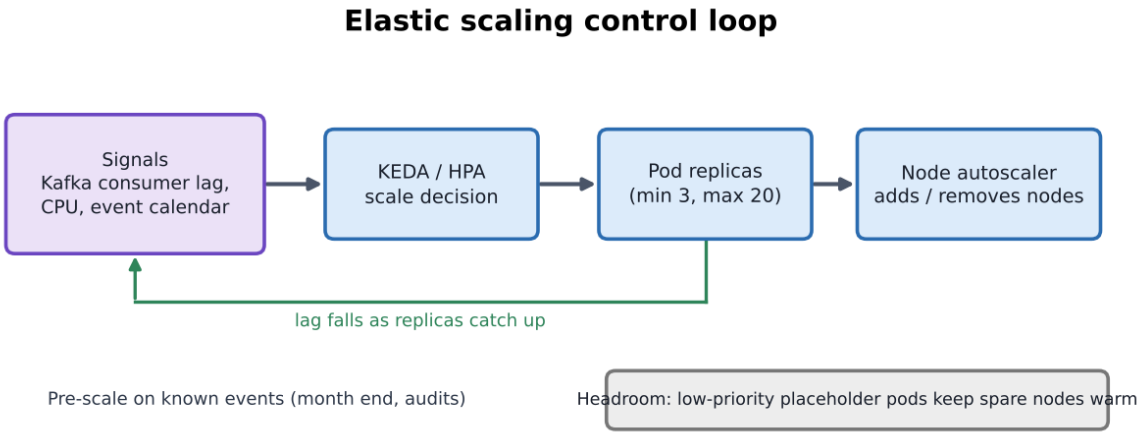


Figure 6: Elastic scaling control loop.

Service	Scaling signal	Min to max (illustrative)	Notes
Ingestion	Request rate, CPU, p99 latency	3 to 12 pods	Minimum of one per AZ. Returns 429 above a safe limit to protect Kafka.
Feature Enrichment	Consumer lag	3 to 12	Parallelism capped by partitions. State is local and rebuilt from Kafka on moves.
Fraud Scoring	Consumer lag, CPU	3 to 20	About 100 TPS per 2-vCPU pod at roughly 50 percent CPU, so 10 pods at 1,000 TPS and 20 at the 2,000 TPS test.
Decision & Alerting	Consumer lag	3 to 6	Very light. Rarely the bottleneck.
Alert Query & Case	Request rate, lag	3 to 8	Read replicas absorb query load.
Notification gateway	Connected clients	3 to 10	Scales with analysts online, not transaction rate.
Archive, Replay	Lag; job queue	2 to 8; 0 to 10	Replay scales to zero when idle and runs on spot capacity.

Scaling down safely. Scale-down is slower than scale-up, uses stabilisation windows to avoid flapping, and relies on graceful shutdown so a pod finishes in-flight work and commits offsets before it exits. Placeholder pods keep spare nodes warm so scale-up does not wait for a new machine to boot.

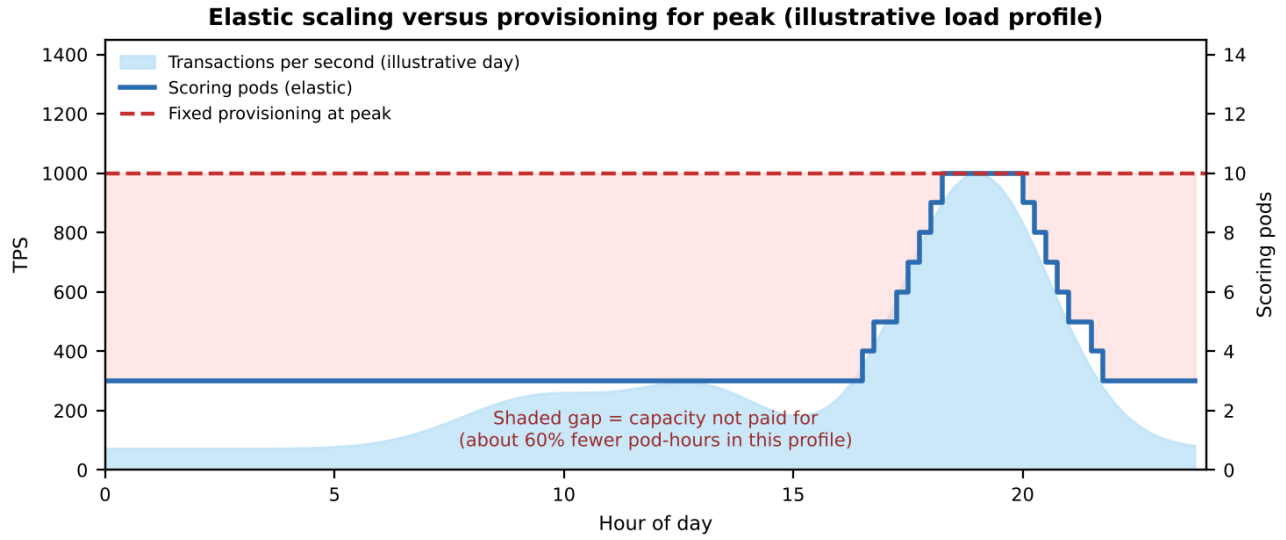


Figure 7: Illustrative day: elastic scoring capacity versus provisioning for peak.

In this illustrative profile (average about 290 TPS, peak 1,000), elastic scaling uses about 96 pod-hours a day against 240 for fixed peak provisioning, roughly **60 percent less compute**, while still meeting peak. The exact saving depends on the real load shape.

12.2 Scaling event messaging

- **Bandwidth is not the constraint.** At peak, about 8 MB per second is produced before replication. A small broker cluster handles this comfortably. The real limit is consumer parallelism, which equals the partition count.
- **Partitions** are over-provisioned up front (48) because adding partitions later breaks key ordering guarantees for in-flight data.
- **Brokers scale horizontally** by adding brokers and rebalancing partitions. This is a planned operation, not an instant one, so the messaging tier is **sized for peak plus headroom and not autoscaled**. It is a small share of total cost.
- **Cost and protection levers:** compression, tiered storage for older data, and per-client quotas so that replay or a misbehaving consumer cannot starve live traffic.

12.3 Scaling analytic workloads

Analytic work (model training, compliance queries, replay, trend reports) is kept off the real-time path entirely. It reads from the archive and lake, never from operational stores, and runs on separate compute:

- **Training and batch analytics** run on ephemeral clusters and spot instances that exist only for the job.
- **Compliance and ad-hoc queries** use a serverless SQL engine over columnar files with per-team workload limits, so one heavy query cannot affect others.
- **Replay** scales on its own node pool and Kafka quotas (Section 8).
- **Common reports** are pre-computed as materialised views so most audit questions never scan raw data.

12.4 Capacity summary

Dimension	Planning figure
Peak and tested load	1,000 TPS peak. Tested to 2,000 TPS (2x headroom)
Events on the critical path	About 4 events per transaction, so roughly 4,000 events per second at peak
Kafka produce rate at peak	About 8 MB/s (about 24 MB/s of replication traffic across brokers)
Scoring capacity	About 10 pods at peak, 20 at the 2x test, minimum 3 (one per AZ)
Alert volume	Roughly 1 to 10 alerts per second at peak (0.1 to 1 percent of transactions)
Analyst concurrency	A few hundred concurrent WebSocket connections. Gateway pods can each hold thousands

13. Resilience

The platform assumes components will fail. This section names the single points of failure, how each is mitigated, and how data is protected and restored.

13.1 Single points of failure and mitigations

Potential single point of failure	Impact if it fails	Mitigation	Residual risk
Kafka cluster	No new events flow. Ingestion cannot publish.	Managed, three AZs, replication factor 3, min in-sync replicas 2, acks=all. Cross-region mirror. Ingestion returns 503 and the source retries.	Region-wide loss: recover in DR region (RPO seconds)
Ingestion service / edge	Bank cannot submit transactions.	Stateless, at least 3 pods across AZs, disruption budgets, load balancer health checks. Source systems buffer and retry (outbox pattern).	Source must implement retry. Agreed in the interface contract
Schema Registry	New schemas cannot be registered.	Replicated (stored in Kafka). Clients cache schemas, so running services are unaffected.	Deployments that change schemas wait
Redis feature store	Enrichment loses features.	Clustered with replicas across AZs. If unavailable, continue with default features and flag events degraded. Rebuild from Kafka.	Lower detection quality while degraded
Model registry / artifact store	Cannot load new models.	Scoring keeps the last-known-good model in memory and on local disk. Durable object storage behind the registry.	Model updates delayed
PostgreSQL (alerts, cases)	Analysts cannot act on alerts.	Managed multi-AZ with automatic failover, point-in-time recovery, cross-region replica. Alerts still appear via the push path.	Brief failover pause (tens of seconds)
OpenSearch	Search degrades.	Multi-AZ. Fully rebuildable from the log. UI falls back to basic queries on PostgreSQL.	Reduced search until rebuilt
Notification gateway	Alerts stop appearing live.	Many stateless pods. Clients reconnect with jitter and resume by sequence number. Polling fallback through the BFF.	Slightly higher alert latency
Archive service	Capture falls behind.	Kafka buffers 7 days. Lag alert at 5 minutes. Reconciliation proves completeness after recovery.	None if recovered inside retention
Service mesh control plane	No config changes.	Multiple replicas. Data plane keeps its last configuration.	Policy changes delayed
Observability stack	Operators lose visibility, not transactions.	Separate failure domain, HA, buffering in collectors, and an independent dead-man's-switch alert.	Reduced insight during an outage
Identity provider	Analysts cannot sign in.	Existing sessions survive. Break-glass access. Detection and archive are unaffected.	New logins blocked
Single region	Total loss on regional outage.	Warm-standby DR region with replicated Kafka, database and archive (Figure 8).	RPO of seconds, RTO of tens of minutes

13.2 Graceful degradation

- **Overload:** ingestion returns 429 with a retry hint rather than accepting work it cannot process. Within the pipeline, Kafka absorbs bursts and autoscaling catches up.
- **Degraded scoring:** if features are incomplete the transaction is still scored and marked degraded. The degraded ratio is an SLO (Section 14.5), so reduced quality is visible, not silent.
- **Load shedding order:** in an extreme surge, non-essential work is shed first (dashboard refreshes, replay, analytics) while detection and capture are protected.
- **Bulkheads:** separate node pools and consumer groups for live, replay and analytic work, so one cannot consume another's capacity.

13.3 Data loss and recovery

The durability rule is simple: **a transaction is acknowledged to the bank only after it is durably written to three replicas.** From that point nothing can lose it silently.

Scenario	Could anything be lost?	How loss is prevented	Recovery
Ingestion crashes before the Kafka write	No. The request was never acknowledged.	Ack only after durable write. Idempotency key.	Source retries safely

Scenario	Could anything be lost?	How loss is prevented	Recovery
Broker or AZ failure	No acknowledged data.	Replication factor 3, min ISR 2, acks=all.	Automatic leader election
Consumer crashes mid-processing	No.	Offsets committed after processing. Idempotent handling.	Message reprocessed
Malformed or poison message	No.	Retry then DLQ.	Fix, then re-drive from the DLQ
Redis data lost	Derived state only.	Features come from events.	Rebuild from Kafka or archive for the feature window (hours)
Alert database corrupted	No business data. Analyst actions are events.	Point-in-time recovery. Alert and disposition events are archived.	Restore, then replay events since the restore point
Archive object damaged	No.	WORM, versioning, cross-region replica, hash manifests.	Restore from replica, verify hashes
Regional disaster	At most the final seconds of asynchronous replication.	Cross-region mirror and replicas.	Fail over. Reconcile against the bank's ledger and re-ingest any gap idempotently

Reconciliation is the safety net. An hourly job compares counts and identifiers between the bank's source ledger, ingestion, and the archive. Any difference raises an alert and is repaired by idempotent re-ingestion. Recovery procedures are rehearsed in scheduled game days, because an untested recovery plan is only a hope.

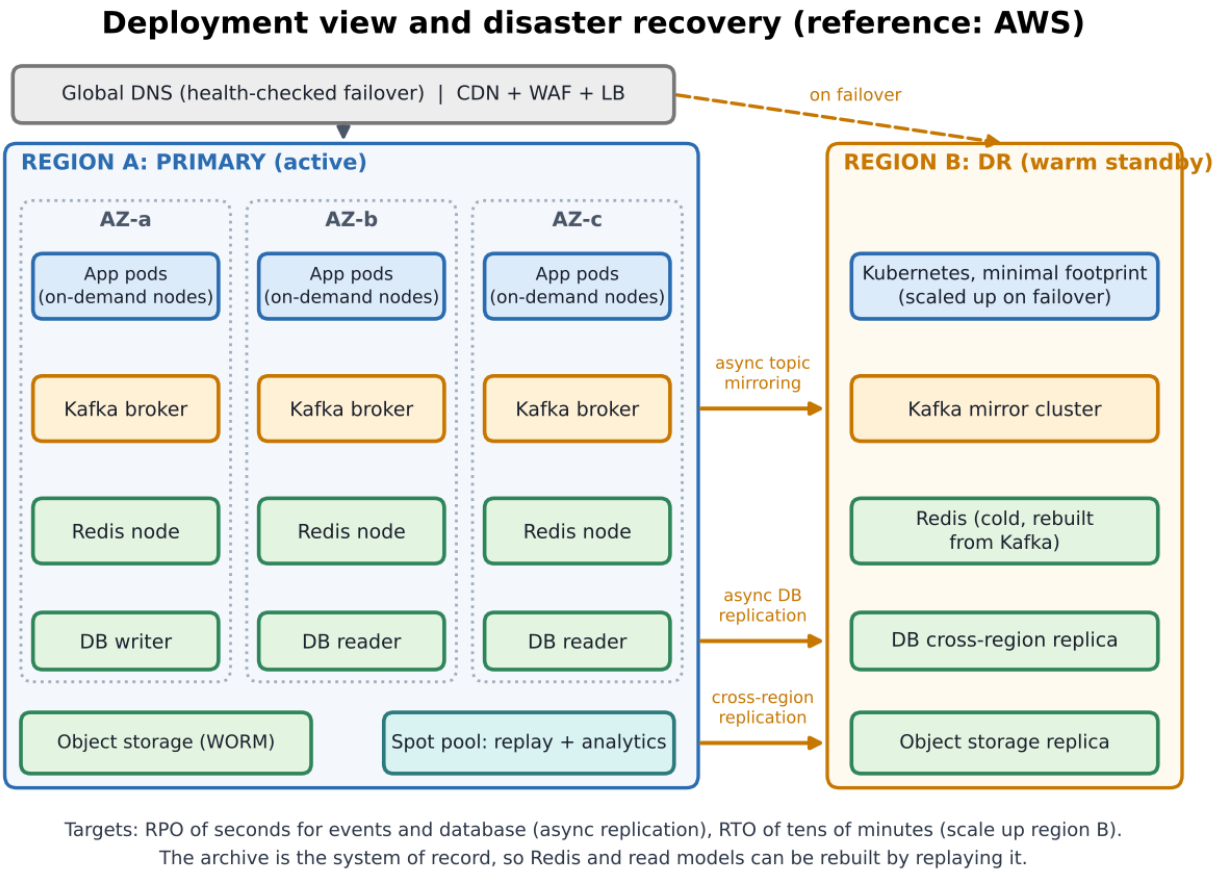


Figure 8: Deployment view and disaster recovery (AWS as the reference platform).

14. Observability

A fraud system that cannot be seen cannot be trusted. Observability here means answering three questions quickly: is the system meeting its objectives, if not why, and is the model still behaving. The design uses open standards (OpenTelemetry) so that the stack stays maintainable and extensible as the platform grows.

14.1 How logs, metrics and traces are collected

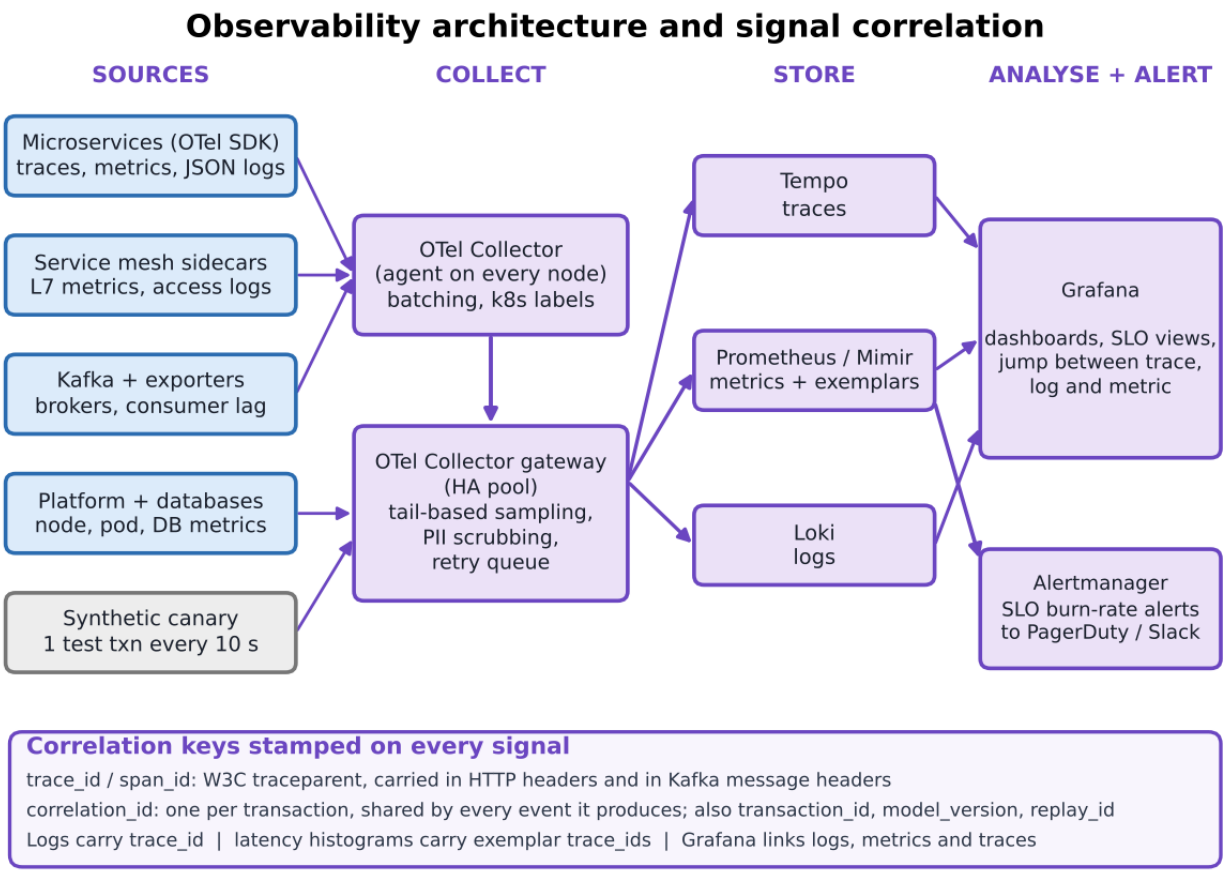


Figure 9: Observability pipeline and the identifiers that correlate every signal.

Signal	Instrumentation	Collection path	Storage and retention
Traces	OpenTelemetry SDK in every service. Mesh adds spans for calls. Trace context is injected into Kafka message headers and extracted by consumers.	SDK, to node agent, to gateway collector (tail sampling), to Tempo	7 days. Keep 100 percent of errors, traces over 1 s, and alert-related traces, plus a 5 percent sample of the rest
Metrics	Application metrics through OpenTelemetry. Kafka broker and consumer-lag exporters. Mesh and platform metrics. Synthetic canary.	Scraped or pushed to collectors, to Prometheus or Mimir	15 days at full resolution, 13 months downsampled
Logs	Structured JSON to stdout. Fields: timestamp, level, service, trace_id, span_id, correlation_id, event_id. Never PII.	Node agent, to gateway (PII scrubbing), to Loki	30 days searchable, 1 year in cold storage

The business audit trail is the event archive (Section 8), not logs or traces. Telemetry may be sampled and expire; compliance data may not.

14.2 How logs, metrics and spans are correlated

- **One trace identifier follows the transaction.** The W3C traceparent header travels in HTTP calls and in Kafka message headers, so a transaction is one trace even though it crosses five asynchronous hops. Batch consumers use span links.
- **Logs to traces:** every log line carries trace_id and span_id. In Grafana, a log line links straight to its trace.
- **Metrics to traces:** latency histograms attach exemplars (sample trace identifiers). A latency spike on a dashboard opens an example slow trace in one click.
- **Transaction-level search:** correlation_id and transaction_id are on logs and spans (never as metric labels, which would explode cardinality), so support can find everything about one transaction.
- **Context labels:** model_version, policy_version and replay_id appear in telemetry, so a quality or latency change can be tied to a specific release.

14.3 Synthetic end-to-end probes

A marked synthetic transaction is sent through the real ingestion path every 10 seconds. It is excluded from analyst queues and reports but measured at every stage. This gives a constant end-to-end latency reading independent of real traffic and detects the dangerous case of a pipeline that silently stops during quiet hours.

14.4 Key metrics and service level objectives

Three metrics matter most for a system like this, because together they cover speed, flow and correctness:

Key metric	Why it matters	How it is instrumented	Alert condition
1. End-to-end detection latency (acceptance to scored event, and to alert on screen)	The legal requirement is real-time detection. This is the customer-visible measure of it.	Histogram computed from event timestamps in Scoring, and from a browser beacon for push-to-render. Exemplars link to traces.	SLO burn-rate alerts (14.5). Also page if the synthetic probe exceeds 3 s twice in a row
2. Consumer lag (offset and time-based, per consumer group)	Lag is the earliest, most direct sign of a bottleneck or outage, and it drives autoscaling.	Kafka lag exporter, plus lag_seconds (now minus timestamp of the oldest unprocessed message).	Warn at lag over 5 s for 2 minutes. Page at over 30 s. Archive lag over 5 minutes
3. Model health (score drift, alert rate, degraded ratio)	A fast system that is quietly wrong is worse than a slow one. Drift and rate shifts reveal both attacks and broken features.	Population Stability Index of the score distribution against the training baseline. Percentage of transactions decided FRAUD per hour. Share of degraded scores. Feature null rate. Analyst-confirmed precision (lagging).	PSI over 0.2 for 1 hour. Alert rate outside 3 standard deviations of the same-hour baseline. Degraded ratio over 0.5 percent

Supporting metrics: request rate, errors and duration for each service; dead-letter queue depth (any message is a warning); under-replicated Kafka partitions; Redis latency and memory; pod restarts and saturation; WebSocket connections and push latency; archive reconciliation difference; analyst queue size and time-to-disposition.

14.5 Service level objectives

Each SLO is specific, measurable and objective, evaluated over a rolling 30-day window.

SLO	Objective	How it is measured (SLI)
SLO 1: Detection latency	99% of transactions have a scored event within 1 second of ingestion acceptance, and 99.9% within 5 seconds.	Scoring service histogram: scored event time minus accepted_at
SLO 2: Alert visibility	99% of fraud alerts are visible in the analyst UI within 3 seconds of ingestion acceptance.	Browser beacon and synthetic canary, correlated by trace_id
SLO 3: Ingestion	99% of POST /v1/transactions requests complete in under 100 ms. 99.9% return a non-5xx response (excluding 429s for clients above agreed rates).	Edge and Ingestion request metrics
SLO 4: Capture completeness	99.999% of accepted transactions are present in the archive within 5 minutes. 100% within 24 hours.	Hourly reconciliation of ingestion count against archive count
SLO 5: Analyst API	99% of GET calls from the analyst UI to the BFF complete in under 300 ms.	BFF request duration histogram
SLO 6: Scoring quality of service	99.5% of transactions each day are scored by the current promoted model with complete features.	Share of events with degraded = false

Error budget example. At 26 million transactions a day, SLO 1 allows 1 percent, about 260,000 transactions a day, to take longer than 1 second. When the budget is being consumed too fast, feature releases pause and reliability work takes priority.

14.6 Alerting strategy

- **Alert on symptoms (SLOs), investigate causes (lag, errors, saturation).** This keeps pages meaningful.
- **Multi-window burn-rate alerts.** Page when 2 percent of the monthly error budget burns in 1 hour (confirmed over 5 minutes). Page when 5 percent burns in 6 hours (confirmed over 30 minutes). Open a ticket when 10 percent burns over 3 days.
- **Routing:** Alertmanager to PagerDuty for pages and Slack for tickets, with runbook links and the relevant dashboard and sample trace in each alert.
- **Dead-man's switch:** an independent monitor pages if the observability pipeline itself stops sending data.

15. Peak usage profiles and compliance events

Production load rarely matches the design profile. Several predictable events stress the platform in ways the 1,000 TPS figure does not capture. The proactive measures below keep the system operable through them.

Event	What changes	Proactive measures
End of financial year and month end	Higher transaction volume. Heavy reporting. Compliance teams run extra queries.	Calendar-driven pre-scaling a day ahead. Raise minimum replicas. Load test at 2x beforehand. Pre-compute reports as materialised views. Change freeze for the surrounding days.
Retail peaks (holiday sales)	Volume above normal peak, with a different mix and more fraud attempts.	Pre-scale from last year's curve plus a margin. Confirm cloud quotas in advance. Verify Kafka headroom. On-call roster doubled.
Compliance audit	Large replays and wide archive queries.	Replay isolated on its own node pool, topics and quotas. Throttled and scheduled off-peak. Query workload limits. Pre-built audit reports. Dedicated read replicas. Early notice from compliance so capacity can be reserved.
Fraud attack or alert storm	Sudden surge of alerts overwhelms analysts and the gateway.	Alert grouping and rate limiting per queue. Severity-based routing. Gateway scales on connections. Rules to auto-escalate to a duty manager when alert rate passes a threshold.
Model or policy rollout during busy periods	A defect would hit peak traffic.	No promotions during freeze windows. Shadow then canary promotion with automatic rollback on SLO regression.
Region or AZ outage at peak	Capacity halves in the surviving zones.	Capacity sized so any two AZs can carry peak. Tested DR failover runbook and quarterly game days.

General controls: game days that deliberately simulate these events, capacity dashboards that show headroom against the 2x target, budget and quota alerts, and runbooks reviewed after each exercise.

16. Deployment in practice and migration

16.1 Reference deployment on AWS

AWS is used as the reference. Each component maps to a managed service, and each mapping introduces provider-specific details the design must respect.

Component	AWS service	Provider-specific adaptation
Kubernetes	Amazon EKS with Karpenter	Plan pod IP capacity (ENI limits). Node start takes minutes, so placeholder pods keep spare capacity warm. Topology spread across three AZs.
Kafka	Amazon MSK (or Confluent Cloud)	Broker count and partition-per-broker limits drive sizing. Broker scaling is an operation, so provision for peak. Use tiered storage. Cross-region replication for DR.
Redis	ElastiCache for Redis (cluster mode, multi-AZ)	Failover takes seconds. Size for the feature window. Not replicated cross-region (rebuilt).
PostgreSQL	Aurora PostgreSQL with Global Database	Use connection pooling. Failover takes tens of seconds.
Search	Amazon OpenSearch Service	Multi-AZ with standby. Snapshots to object storage.
Archive	S3 with Object Lock (compliance mode), replication, lifecycle tiers	Store in a separate, locked-down security account. Lifecycle to infrequent-access and archive tiers.
ML and analytics	SageMaker, Athena and Glue, EMR Serverless	Spot capacity for training. Per-team workgroup limits.
Edge and DNS	CloudFront, AWS WAF, load balancer, Route 53 failover	Health-checked failover routing to the DR region.
Observability	Managed Prometheus and Grafana, or self-hosted Tempo and Loki	Keep the collector pipeline vendor-neutral so the backend can change.
Keys and secrets	KMS (separate keys for archive), Secrets Manager	Key policies restrict who can decrypt the archive.

Account structure and cost. Separate accounts for production workloads, the security and archive vault, shared networking, and non-production, with organisation-level guardrails. Baseline compute is covered by committed-use discounts, burst and batch use spot, and budgets with alerts guard against runaway scaling. The same design maps to other providers (for example AKS with a Kafka-compatible service and immutable blob storage, or GKE with locked buckets).

16.2 Migration from the legacy batch system

Replacing a live fraud control is risky, so the cutover is incremental and reversible (the strangler fig approach).

Phase	Activity	Exit criteria
0 Foundations	Landing zone, Kafka, observability, CI/CD, security reviews.	Platform passes security and resilience review
1 Shadow mode	Mirror live transactions to the new platform. No analyst impact. Compare decisions with the legacy system and with confirmed fraud outcomes.	Detection at least equal to legacy. SLO 1 and 4 met for 30 days
2 Analyst pilot	A small analyst team works from the new UI alongside the legacy tools.	Analyst acceptance. Alert quality validated
3 Gradual cutover	Move traffic by product line or segment. Legacy remains as fallback.	DR test passed. Replay verified by compliance
4 Decommission	Retire the legacy system. Retain its historical results for the required period.	Formal sign-off from risk and compliance

17. Critical evaluation: trade-offs and risks

Every significant choice carries a cost. In financial services the cost of error is high, so trade-offs are stated explicitly together with the reason each is judged worthwhile.

Decision	Benefit	Cost or risk	Why it is worthwhile, and mitigation
Detect after authorisation (advisory) instead of blocking inline	Fraud-system issues cannot stop payments. Simpler availability model.	A fraudulent payment may complete before an alert. Not suitable if the bank needs pre-authorisation blocking.	Matches the stated requirement (analyse, classify, alert). A synchronous decision API can later reuse the scoring components, at the cost of coupling payment availability to this system.
Event-driven with eventual consistency	Decoupling, burst absorption, replay.	The UI and read models lag the log by milliseconds to seconds. Harder end-to-end reasoning.	Alerts are advisory. Analyst actions use strongly consistent writes. Tracing and lag metrics make flow visible.
Kafka rather than a simple queue	Replay, ordering per key, many consumers, high throughput.	Operational complexity and a critical shared dependency.	Replay and ordering are core requirements. Managed service and DR mirror reduce operational load.
At-least-once plus idempotency, not exactly-once everywhere	Simpler, faster, proven.	Duplicates must be handled by every consumer.	Idempotent design is a standard pattern. Exactly-once is used only where a stream processor needs atomicity.
Ten components rather than a monolith	Independent scaling, fault isolation, clear ownership.	More deployments, more network paths, more to observe.	Scaling profiles genuinely differ (CPU, I/O, connections). Mesh and common templates reduce the per-service cost.
Service mesh	mTLS and policy across all services without code.	Latency and resource overhead. Another control plane.	Zero-trust is expected in banking. Overhead is small compared with the latency budget and can be reduced with lighter modes.
Messaging tier sized for peak, not autoscaled	Predictable, safe under surge.	Pays for peak broker capacity at all times.	Brokers are a small share of cost. Elasticity is concentrated in compute, where the savings are.
Immutable archive	Strong audit evidence, tamper resistance.	Conflicts with erasure rights. Storage cost over 7 years.	Tokenisation with crypto-shredding. Storage tiering. Legal sign-off on the approach.
Model embedded in the scoring service	No network hop, lower and steadier latency, simpler operations.	Model and service release are coupled. Large models need larger pods.	Models are small trees today. A separate model server is a straightforward later change if GPU models arrive.
Warm-standby DR rather than active-active	Much lower cost and complexity.	Recovery takes tens of minutes. Small possible loss of final seconds of replication.	Matches an advisory system. Reconciliation against the source ledger closes the gap.
High-confidence threshold for alerts	Limits false positives and analyst fatigue.	Some real fraud falls in the REVIEW band and is seen later.	Threshold is a business policy, tunable without redeploying a model. A REVIEW queue covers the middle band.
Tail-based trace sampling	Keeps every important trace at manageable cost.	Normal traces are only sampled. The sampling gateway needs memory.	Errors, slow and alert traces are always kept. Compliance relies on the archive, not traces.
Managed cloud services	Faster delivery, less undifferentiated operations.	Provider lock-in for some services.	Open standards at the core (Kafka, Kubernetes, OpenTelemetry, PostgreSQL) keep exit options open.

17.1 Principal risks and challenges

- **Model drift and governance:** models degrade as fraud patterns change. Mitigated by drift monitoring, shadow and canary promotion, recorded explanations, and a documented approval process.
- **Alert fatigue and false positives:** too many alerts reduce analyst effectiveness. Mitigated by calibrated thresholds, grouping, and feedback labels that improve the model.
- **Hot partitions:** very busy accounts can overload one partition. Monitored through partition skew and mitigated by key design.
- **Schema evolution over seven years:** consumers and replay must read old events. Mitigated by compatibility rules in CI and versioned topics.
- **Operational skills:** Kafka, Kubernetes and the mesh require expertise. Mitigated by managed services, platform templates and runbooks.
- **Regulatory variation:** retention, privacy and residency differ by jurisdiction. Requires early engagement with compliance and legal.
- **Replay determinism:** only reproducible if model versions, policy versions and feature logic are all versioned and recorded. This is tested as part of acceptance.

18. Conclusion and next steps

The proposed platform replaces the overnight batch with a real-time, event-driven pipeline that scores every transaction within a second, shows alerts to analysts within seconds, and keeps a complete immutable record that can be replayed for auditors. It scales with demand, tolerates the loss of a zone or a dependency, and exposes enough telemetry to be run against measurable objectives.

Recommended next steps before full build:

- Confirm the volume, retention and inline-blocking assumptions with the business, risk and compliance.
- Build a thin vertical slice (ingest, enrich, score, alert) and load test to 2,000 TPS to validate the capacity figures.
- Prove replay determinism with compliance using real historical samples.
- Run a first disaster-recovery game day, and start shadow mode against the legacy system.

Appendix A: Requirements traceability

Requirement	Where addressed
Functional decomposition: at least 3 microservices; function, inputs and outputs; inter-service communication and coupling	Section 5 (service table, 5.1, 5.2). Figure 1
Key events (what, when, why); publishers and consumers; deviations from event-driven; a trigger for each service	Sections 6.1 to 6.6 and Section 5. Figure 2
Three key metrics to monitor; SLOs	Sections 14.4 and 14.5
Observability: how logs and metrics are collected; how logs, metrics and spans are correlated	Sections 14.1 to 14.3. Figure 9
Single points of failure, mitigations, data loss and restoration	Sections 13.1 and 13.3. Figure 8
Scaling microservices, analytic workloads and event messaging	Sections 12.1, 12.3 and 12.2. Figures 6 and 7
Business problems decomposed into technical objectives	Section 2.1
Component-level outline of how problems are solved	Section 4
Rationale for approaches; patterns with cost-benefit	Sections 3 and 17
High-level guidance rather than implementation detail	Throughout: patterns and structure, not code
Solution architecture diagram; event flow diagram; end-to-end transaction flow diagram	Figures 1, 2 and 3
Additional diagrams	Figures 4 to 9 (replay, frontend, scaling, elasticity, deployment, observability)
Frontend, service mesh, BFF, caching, event-driven contributions	Sections 9, 10, 3 and 6
Technology stack with justification	Section 11
Realistic deployment on a compute provider	Section 16.1
Heavy-load events such as audits and end of financial year	Section 15

Appendix B: Glossary

Term	Meaning
BFF	Backend for Frontend: a service that shapes APIs for one specific user interface
CQRS	Command Query Responsibility Segregation: separate models for writing and reading data
DLQ	Dead-letter queue: where messages that repeatedly fail processing are parked
PSI	Population Stability Index: a measure of how much a distribution has drifted
RPO / RTO	Recovery point objective (data loss tolerated) and recovery time objective (time to restore service)
SLI / SLO	Service level indicator (a measurement) and objective (the target for it)
TPS	Transactions per second
WORM	Write once, read many: storage that cannot be altered or deleted during retention